STAIRMIGRATION

Product Requirements Document

AI Software Migration Engineer

Evidence-backed migration through code analysis, structure-aware retrieval, reviewable patches, and isolated verification.

MVP MIGRATION PATH

Spring Boot 2.7 → 3.0

Maven / Java 17 / Human-reviewed patches

PROPOSED MVP

01 November 2026

VERSION 1.0

Product + engineering specification

Read this specification

A practical contract for the first migration workflow. The product preserves repository context, grounds proposed changes in versioned documentation, and makes verification visible before export.

Section	What it decides
01 Product + research	Problem, product boundaries, and the STAIR adaptation
02 Users + success	User outcomes and measurable release gates
03 MVP scope	Supported patterns and explicit exclusions
04 End-to-end experience	Review flow and functional requirements
05 System architecture	Analysis, retrieval, patching, and verification
06 Data model	Versioned entities and provenance
07 API surface	Run, findings, patches, and artifacts
08 Patch + verification policy	Controls for generation and sandbox execution
09 Evaluation	Quality, observability, and retention
10 Risks + release	Controls and pilot exit criteria
11 Delivery gates + sources	Mandatory manual checks and source register

Document use

This PDF is the designed reading edition. The repository includes an editable Markdown PRD and the detailed phased development plan. Implementation is not included in this initial repository.

Target context

Spring Boot 2.7 to 3.0 is a bounded historical migration target for implementation and evaluation. An actual production destination requires a separate support and compatibility decision.

01 Product + research

Product summary

AI Software Migration Engineer (ASME) is a developer tool that analyzes a Spring Boot 2.7 repository, produces evidence-backed migration patches for Spring Boot 3.0, applies selected patches in an isolated copy, and verifies the result with the repository's build and tests. It is an assistive migration system: it must show the exact source and rule behind each proposed change, never silently modify a user checkout, and never report success when verification has not run or failed.

The MVP combines static code analysis with a **STAIR-inspired structure-aware retrieval layer**. Repository modules, packages, classes, build files, configuration files, and symbols are represented as a navigable hierarchy rather than as flat chunks alone. Retrieval uses this hierarchy alongside lexical and embedding search to gather the smallest defensible evidence set for an issue and a version-specific migration recipe.

Why this problem needs an AI-assisted workflow

Conventional codemods work well for deterministic renames, but do not explain ambiguous changes, connect build failures to the relevant framework documentation, or safely decide which of several context-dependent fixes applies. General RAG chatbots can explain changes but commonly lose repository context, retrieve stale documentation, and cannot prove that a proposed patch builds. ASME joins deterministic analyzers, version-pinned evidence retrieval, constrained patching, and sandboxed verification.

Research grounding and product decision

IBM Research's STAIR paper proposes generating an addressable leaf from a document's table of contents, preserving global structure that fixed-size chunking loses. On its SearchTome benchmark (18 books across six domains), it reports stronger Recall@1 than its evaluated baselines. Paper.

What we adopt now: explicit hierarchy, section/module paths in retrieval, leaf-level evidence citations, and evaluation against flat/hybrid baselines.

What we do not claim or ship in the MVP: a fine-tuned parametric STAIR/DSI index, learned document identifiers, or IBM-equivalent quality. STAIR needs a complete, coherent table of contents and supervised training; its benchmark consists of structured books, not evolving polyglot codebases. The paper's retrieval metric does not establish patch correctness, security, build success, or generalization to repositories with generated code, weak module boundaries, or incomplete documentation. A constrained ToC generator can prevent invalid identifiers, but it cannot make an incorrect valid section useful. The MVP therefore retains inspectable indexes and a hybrid fallback; a trainable hierarchy router is a later experiment only if it beats the baseline on held-out fixtures.

02 Users + success

Users and jobs to be done

User	Job	Primary outcome
Application maintainer	Modernize a service without reading every migration guide	A reviewed patchset with verification evidence
Platform engineer	Move many similar services with consistent policy	Reusable rules, audit trail, and measurable reliability
Reviewer	Assess whether an AI patch is safe	Small changes, source links, diffs, and test logs
AI engineer	Improve retrieval and repair quality	Traceable evaluation data and failure categories

Goals and measurable outcomes

1. Ingest a local Maven repository and create a versioned analysis run in under five minutes for a fixture-sized repository (<= 25k non-generated LOC).
2. Detect the supported Spring Boot 2.7 → 3.0 migration patterns with source locations, confidence, and cited official documentation.
3. Produce a patch plan before changing any code; require explicit user selection to apply it.
4. Apply selected changes only in a disposable workspace and run an allowlisted build/test command under time, CPU, memory, and network controls.
5. Report the verification state precisely: **NOT_RUN**, **PASSED**, **FAILED**, **TIMED_OUT**, or **BLOCKED**.
6. Track retrieval evidence and patch outcomes so the team can compare hierarchy-aware hybrid retrieval with flat hybrid retrieval.

Initial success thresholds

Measure	MVP gate
Supported-pattern detection recall on curated fixtures	>= 90%
Unsupported-pattern false “auto-fixable” rate	0% on curated fixtures
Source-location precision for reported findings	>= 95%
Patch applies cleanly to supported fixtures	>= 90%
Verified build/test success after accepted patch	>= 80% of supported fixtures
Every generated patch hunk has evidence IDs	100%
Regression: already-3.0 fixture receives mutations	0

These are release gates, not claims about production repositories. A result that has not passed sandbox verification is labeled “proposed,” never “migrated.”

03 MVP scope

In scope

- Local Git or directory input; Maven wrapper or installed Maven.
- Java 17 baseline; Maven **pom.xml**; single-module and simple multi-module repositories.
- Spring Boot 2.7.x detected from parent/BOM/properties; target Spring Boot 3.0.x.
- Versioned corpus of official Spring Boot 3.0 migration guidance, Spring Framework 6 guidance, Jakarta EE API changes, and project-local migration rules. The initial corpus pins a specific revision of Spring's 2.7 → 3.0 migration guide and the 3.0.0 reference documentation, rather than indexing a moving “latest” page. Each document is recorded with publisher, URL, version/revision, fetch time, checksum, and license/usage note.
- Hybrid retrieval: BM25/lexical + embedding similarity + structure score; diversity/reranking across repository and documentation evidence.
- Deterministic findings and patch recipes for: Boot version/property updates; Java 17 requirement; selected **javax.*** → **jakarta.*** imports; explicit Maven dependency coordinate changes where an approved recipe exists; and deleted/renamed properties that have exact official guidance.
- LLM-assisted explanation and unified-diff generation constrained to retrieved evidence and a selected recipe.
- Disposable sandbox verification: **./mvnw -B test** when wrapper is present, otherwise a configured **mvn -B test**; compilation-only fallback is explicitly labeled.
- JSON/REST API and a minimal local web UI or CLI for review.

Explicitly out of scope

- Gradle, Kotlin/Groovy, Android, Spring Boot > 3.0, or automatic major-version chains.
- Database schema/data migrations, cloud/IaC migration, production deployment, dependency upgrades unrelated to the target migration, and publishing changes to a remote repository.
- Automatic fixes to Spring Security configuration, custom serialization behavior, reflection-heavy frameworks, generated sources, or unresolved compile errors. These can be reported as manual-review findings.
- Executing arbitrary project scripts, Docker daemon access, host-network access, secrets, or mutation of the user’s checkout.
- Fine-tuning/retraining a retrieval LLM for each repository.

04 End-to-end experience

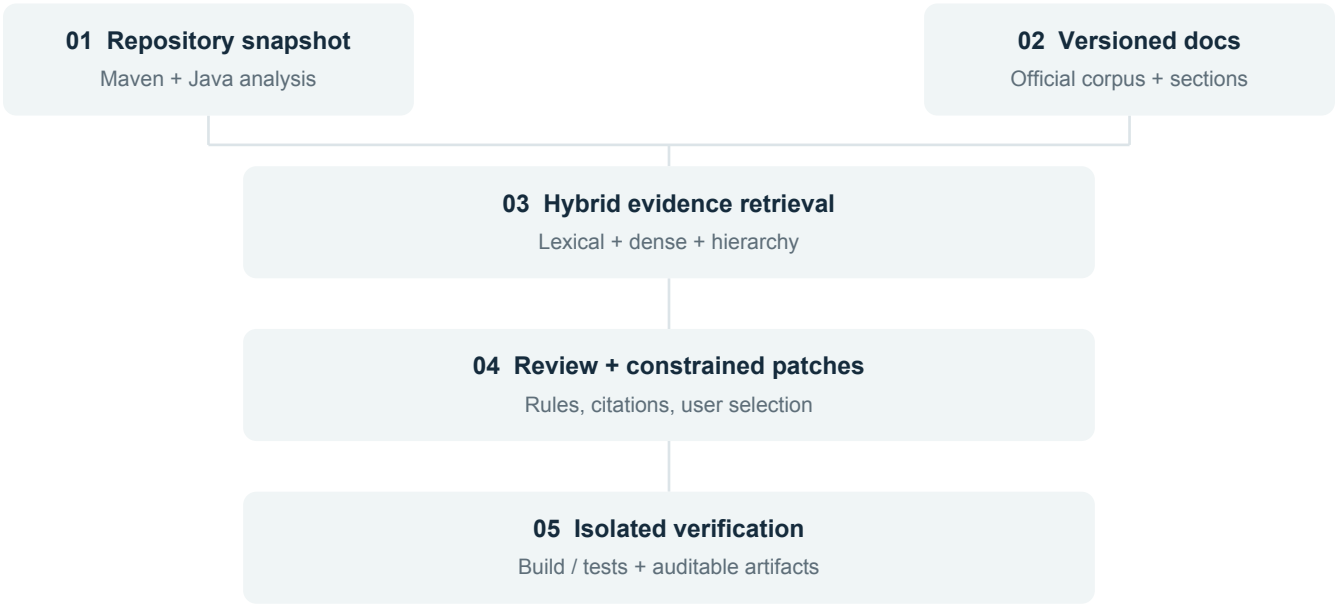
4.1 End-to-end flow

1. User creates a migration run with a repository path and target **spring-boot:3.0.x**.
2. System snapshots the repository's commit/tree hash and validates eligibility (Maven, Boot 2.7, Java baseline, size/policy).
3. Ingestion builds the code graph and document corpus index; it excludes generated/build/vendor directories using policy.
4. Analyzers emit deterministic findings. Retrieval attaches repository context and version-pinned official-doc evidence.
5. User sees a grouped plan: finding, impact, candidate patch, confidence, citations, and verification command.
6. User selects patches. System creates a new disposable workspace, generates a unified diff, validates it applies, and saves its provenance.
7. System executes only the configured verifier in the sandbox and returns status, logs, changed files, remaining findings, and rollback instruction.
8. User may export the verified patch or discard the run. Export is a diff/branch created from the sandbox, never a direct write to the source checkout.

4.2 Requirement detail

ID	Requirement	Acceptance condition
FR-01	Eligibility	Reject/non-destructively label unsupported inputs before analysis.
FR-02	Structure extraction	Persist repo → module → package/path → file → symbol and doc → version → section → passage relations.
FR-03	Hybrid retrieval	Return top evidence with lexical, semantic, and structural score components plus stable IDs.
FR-04	Official-doc governance	A patch cannot use a doc outside the version-pinned allowlist unless marked "local rule/manual review."
FR-05	Findings	Each finding includes rule ID, severity, location, code excerpt hash, rationale, evidence, and confidence.
FR-06	Patching	Generate/apply only user-selected recipe IDs; reject edits outside allowed files/lines or with invalid diff.
FR-07	Verification	Run in a fresh sandbox with resource limits, no secrets, no host network, and captured logs.
FR-08	Auditability	Persist immutable run inputs, corpus version, retrieval trace, model/prompt version, diff checksum, and verifier outcome.
FR-09	Safe recovery	A failed apply/verification leaves the original path byte-for-byte unchanged and preserves a discardable sandbox.

05 System architecture



Service boundaries: the orchestration API has no write access to the source repository; the worker may write only to the run workspace and artifact store; the verifier receives a read-only dependency cache and an empty secret environment. Start as a modular monolith with separate process/container boundaries for verifier execution, not as microservices.

Retrieval design

1. Extract structural paths, e.g. **repo / billing-service / src/main/java / ... / InvoiceController#submit** and **Spring Boot 3.0 migration guide / Jakarta EE 9 / Servlet API**.
2. Index leaf passages with ancestor-title/path text; retain parent/child and file/symbol edges.
3. For a finding, retrieve lexical candidates (exact identifiers/property names), semantic candidates (issue intent), and structural neighbors (same module/file/symbol and parent doc section).
4. Fuse rankings with weighted reciprocal-rank fusion; rerank with a small cross-encoder or LLM only over the compact candidate set.
5. Require a recipe to cite an exact official passage or a local policy. If evidence conflicts or score margins are low, route to manual review.

The A/B evaluation must compare: (A) flat BM25+dense baseline, (B) hybrid with heading/path text, and (C) hierarchy-neighbor expansion. Do not call C “STAIR” unless a distinct trained constrained generative router is implemented and evaluated.

06 Data model

Entity	Key fields
MigrationRun	**id**, source tree hash, target, status, policy version, corpus version, timestamps
RepositorySnapshot	root fingerprint, commit/ref, Maven coordinates, Java version, module graph, exclusion policy
CodeNode	**id**, run ID, parent ID, type, path, symbol, byte/line range, content hash, generated flag
DocArtifact	**id**, corpus version, publisher, URL, framework/doc version, fetched at, checksum, license note
DocNode	**id**, artifact ID, parent ID, heading path, section anchor, text hash, passage text
Finding	**id**, run ID, rule ID, severity, status, location, evidence IDs, confidence, manual-review reason
Recipe	**id**, supported versions, preconditions, patch template/transform, verifier expectations, rollback notes
RetrievalTrace	query hash, candidate IDs, per-channel ranks/scores, selected evidence IDs, retriever version
PatchSet	**id**, run ID, selected finding IDs, unified diff, diff checksum, generated-by model/prompt version, apply status
Verification	patchset ID, command allowlist ID, image/toolchain hash, limits, status, exit code, log artifact IDs

Use append-only run events for state transitions. Content blobs live outside the relational store, addressed by checksum. Redact secrets from configuration before model context or artifact persistence.

07 API surface

Method and route	Purpose
POST /v1/runs	Create run: source reference, target, policy/profile.
GET /v1/runs/{runId}	Run status and high-level summary.
POST /v1/runs/{runId}/analyze	Start/continue ingestion and analysis; idempotent.
GET /v1/runs/{runId}/findings	Paginated findings, filters, evidence summary.
GET /v1/findings/{findingId}/evidence	Retrieval trace and cited passages.
POST /v1/runs/{runId}/patchsets	Create patchset from explicit selected finding IDs.
POST /v1/patchsets/{patchSetId}/apply	Apply only in a new sandbox workspace.
POST /v1/patchsets/{patchSetId}/verify	Queue allowlisted verifier.
GET /v1/patchsets/{patchSetId}/artifacts	Diff, redacted logs, reports, export metadata.
POST /v1/runs/{runId}/discard	Delete disposable workspace/artifacts per retention policy; never source repo.

All mutation routes require an idempotency key. Every response returns a **runId**, **status**, and correlation ID. Authentication may be local-only for MVP, but authorization boundaries must be designed before multi-user deployment.

08 Patch + verification policy

Patch guardrails

- Rule-first: deterministic recipes are preferred. The model may propose a diff only within file/range allowlists supplied by the recipe.
- Evidence-first: at least one official 3.0-versioned passage (or explicit local-policy exception) must support each behavior-affecting change.
- Minimality: do not reformat unrelated files, reorder dependencies, or upgrade unrelated libraries.
- Confidence is not a safety guarantee. Low-confidence or conflicting evidence produces a review task, not an automated patch.
- Validate diff syntax, touched paths, line anchors, and no binary/secret/generated files before apply.

Sandbox specification

- Fresh working copy from immutable snapshot; source repository mounted read-only.
- Rootless container/VM; no network by default; no Docker socket; non-root UID; read-only base image.
- CPU, memory, PID, disk, and wall-clock ceilings; kill and label **TIMED_OUT** on breach.
- Read-only prewarmed Maven cache; dependencies unavailable from cache cause **BLOCKED_DEPENDENCY** rather than network access.
- Capture structured test results and redacted stdout/stderr. A passing process exit alone is insufficient when Surefire reports failures.

09 Evaluation + telemetry

Maintain a frozen fixture corpus with ground-truth findings, approved diffs, expected build/tests, and unsupported cases. Split by repository-not by file-to avoid structural leakage. A run records retrieval, planning, patching, and verification separately so an apparent end-to-end failure can be assigned to the right stage.

Layer	Offline measure	Release gate
Code analysis	precision/recall per rule	meet §4 threshold; no missed critical supported finding
Retrieval	Recall@k and MRR for gold doc + code evidence	hierarchy variant must not regress baseline by >2 points; promote only with statistically reviewed gain
Evidence	citation correctness / version match	100% sampled patch evidence valid and version-pinned
Patching	apply success, diff minimality, human accept rate	no edits outside recipe allowlist
Verification	build/test pass, timeout/block rate	explicit status for 100% of executed runs
Safety	original-worktree hash unchanged; secret scan	100% on every automated E2E test

No production training occurs from user source without explicit opt-in. Store only redacted, retention-limited telemetry; fixture data is the default evaluation corpus.

10 Risks + release

Risks and decisions

Risk	Control / decision
Stale or mixed-version docs	Immutable corpus releases; URL/version/checksum shown in UI; approved-source allowlist.
Bad hierarchy in code/docs	Fallback hybrid retrieval; parser confidence; flat baseline maintained; manual review on ambiguity.
Model invents a credible change	Recipe and evidence gates; constrained diff; sandbox verification; never direct apply.
Sandbox escape or supply-chain code	Rootless isolated runner, no network/secrets/socket, quotas, patched base image, short retention.
Maven builds need downloads	Prewarm pinned cache; classify as blocked; do not silently enable egress.
Fixture overfitting	Hold out repositories and mutation variants; add real anonymized failures only with consent.
Spring 3.0 breadth	Narrow rule set; explicit unsupported taxonomy; defer Security/observability/manual semantic changes.

MVP exit criteria

The MVP can be piloted only when all vertical-slice manual-flow gates in the companion development plan pass, the §4 thresholds are met on the frozen fixture set, every patch is auditable and sandbox-verified or clearly not verified, and the rollback guarantee has been tested. A pilot remains read-only to the user's original repository and requires human export/review before any merge.

11 Delivery gates + sources

Mandatory manual end-to-end checkpoints

Each major vertical slice ends with a recorded manual run. The development plan supplies exact fixtures, actions, expected results, pass/fail criteria, and rollback instructions. A failed gate blocks progression.

Gate	Manual proof required
P0 / Safety baseline	Create runs for clean, unsupported, and malicious fixtures. Prove source immutability and execution boundaries.
P1 / Analysis	Validate findings, source locations, module ownership, Security abstention, and already-target no-op behavior.
P2 / Evidence	Open cited passages and retrieval traces; test missing evidence and wrong-version rejection.
P3 / Patch selection	Inspect every hunk, selected finding, and citation. Prove only approved sandbox changes are applied.
P4 / Full verification	Run analyze → select → patch → apply → verify → export/discard. Confirm pass, failure, and dependency-blocked states.
P5 / Pilot drill	Repeat the complete workflow on all fixtures and a held-out repository; prove recovery and retention behavior.

Source register

[1] Kumar et al. STAIR: STructure Aware Information Retriever. Research basis for preserving document hierarchy. Open source

[2] Spring Boot 3.0 Migration Guide. Official migration guidance, pinned revision eeac953. Open source

[3] Spring Boot 3.0.0 reference: Getting Started. Java/platform baseline. Open source